Google Skills

Core concepts

1. SequentialAgent

From ADK documentation

“A SequentialAgent executes a series of sub-agents one after another in a predetermined order.

How it works:

Python

```python
from google.adk.agents import LlmAgent, SequentialAgent

# Step 1: Collect input
intake = LlmAgent(
    model='gemini-2.5-flash',
    name='intake',
    output_key='user_request',  # Saves output to state
    instruction='Extract the user request'
)

# Step 2: Process (uses Step 1's output)
processor = LlmAgent(
    model='gemini-2.5-flash',
    name='processor',
    output_key='result',
    instruction='Process this request: {user_request}'  # Reads from state
)

# Step 3: Respond (uses Step 2's output)
responder = LlmAgent(
    model='gemini-2.5-flash',
    name='responder',
    instruction='Create final response based on: {result}'
)

# Sequential: runs in order 1 → 2 → 3
root_agent = SequentialAgent(
    name='pipeline',
    sub_agents=[intake, processor, responder]
)
```

Key points:

- Agents run **in order** (list order)
- Use `output_key` to save output to state
- Use `{key}` to read previous output from state
- Each agent **waits** for the previous one to finish

When to use: Fixed order pipelines, data dependencies between steps.

The following diagram shows how SequentialAgent executes its sub-agents in a strict, ordered sequence. Each step must complete before the next one begins:

```
None
graph LR
    S1[Step 1] --> S2[Step 2] --> S3[Step 3]

    style S1 fill:#e1f5ff
    style S2 fill:#e1f5ff
    style S3 fill:#e1f5ff
```

SequentialAgent: Each step runs after the previous one completes.

Reference: ADK Docs - Sequential Agents

2. ParallelAgent

From ADK documentation

“A ParallelAgent executes multiple sub-agents concurrently.
All sub-agents start at approximately the same time.”

How it works:

```python
from google.adk.agents import LlmAgent, ParallelAgent

# Source 1
researcher_a = LlmAgent(
    model='gemini-2.5-flash',
    name='researcher_a',
    output_key='research_a',  # Each branch needs distinct key
    instruction='Research topic A'
)

# Source 2
researcher_b = LlmAgent(
    model='gemini-2.5-flash',
    name='researcher_b',
    output_key='research_b',  # Different key
    instruction='Research topic B'
)

# Parallel: both run at the same time
root_agent = ParallelAgent(
    name='parallel_research',
    sub_agents=[researcher_a, researcher_b]
)
```

Key points:

- Agents run **at the same time** (concurrent)
- Each branch needs **a distinct** `output_key` (to avoid conflicts)
- Waits for **all branches** to complete
- **Faster** than running sequentially

When to use: Independent tasks, gathering data from multiple sources, performance optimization.

The following diagram shows how ParallelAgent executes all its sub-agents simultaneously. All branches start at the same time and run concurrently:

```
graph TB
    START[Start] --> P1[Branch A]
    START --> P2[Branch B]
    START --> P3[Branch C]
    P1 --> END[All Complete]
    P2 --> END
    P3 --> END

    style P1 fill:#eeffee
    style P2 fill:#eeffee
    style P3 fill:#eeffee
```

ParallelAgent: All branches run at the same time, completing faster than sequential execution.

Reference: ADK Docs - Parallel Agents

3. LoopAgent

From ADK documentation

"A LoopAgent repeatedly executes a sequence of sub-agents until a maximum iteration count is reached."

How it works:

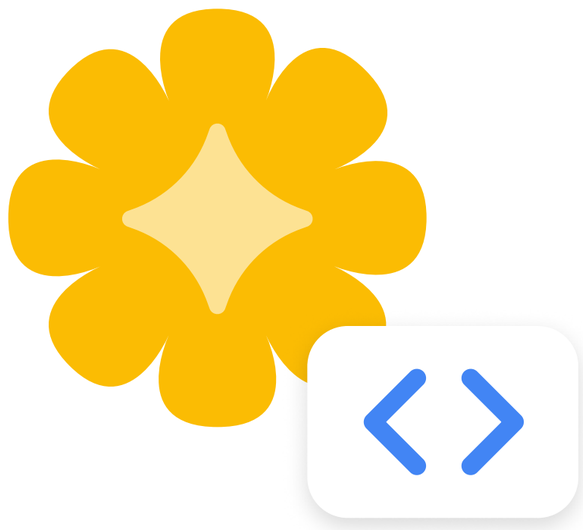
Python

```python
from google.adk.agents import LlmAgent, LoopAgent

# Check quality
checker = LlmAgent(
    model='gemini-2.5-flash',
    name='checker',
    output_key='feedback',
    instruction='Rate the content quality 1-10. Give feedback.'
)

# Improve based on feedback
improver = LlmAgent(
    model='gemini-2.5-flash',
    name='improver',
    instruction='Improve the content based on: {feedback}'
)

# Loop: repeat up to 3 times
root_agent = LoopAgent(
    name='refinement_loop',
    sub_agents=[checker, improver],
    max_iterations=3  # Safety limit
)
```

Key points:

- Agents run **repeatedly** in a loop
- `max_iterations` prevents infinite loops (always set this!)
- Good for **refinement** - check quality, improve, repeat
- Each iteration can use output from previous iteration

When to use: Iterative improvement, quality validation loops, retry patterns.



The following diagram shows how LoopAgent repeatedly executes its sub-agents until a condition is met or the maximum iterations is reached:

```
None
graph LR
    L1[Check] --> L2[Improve]
    L2 --> L1
    L1 -.-> DONE[Done when max_iterations reached]

    style L1 fill:#ffeb99
    style L2 fill:#ffeb99
```

LoopAgent: Agents repeat in a cycle until max_iterations is reached or a termination condition is met.

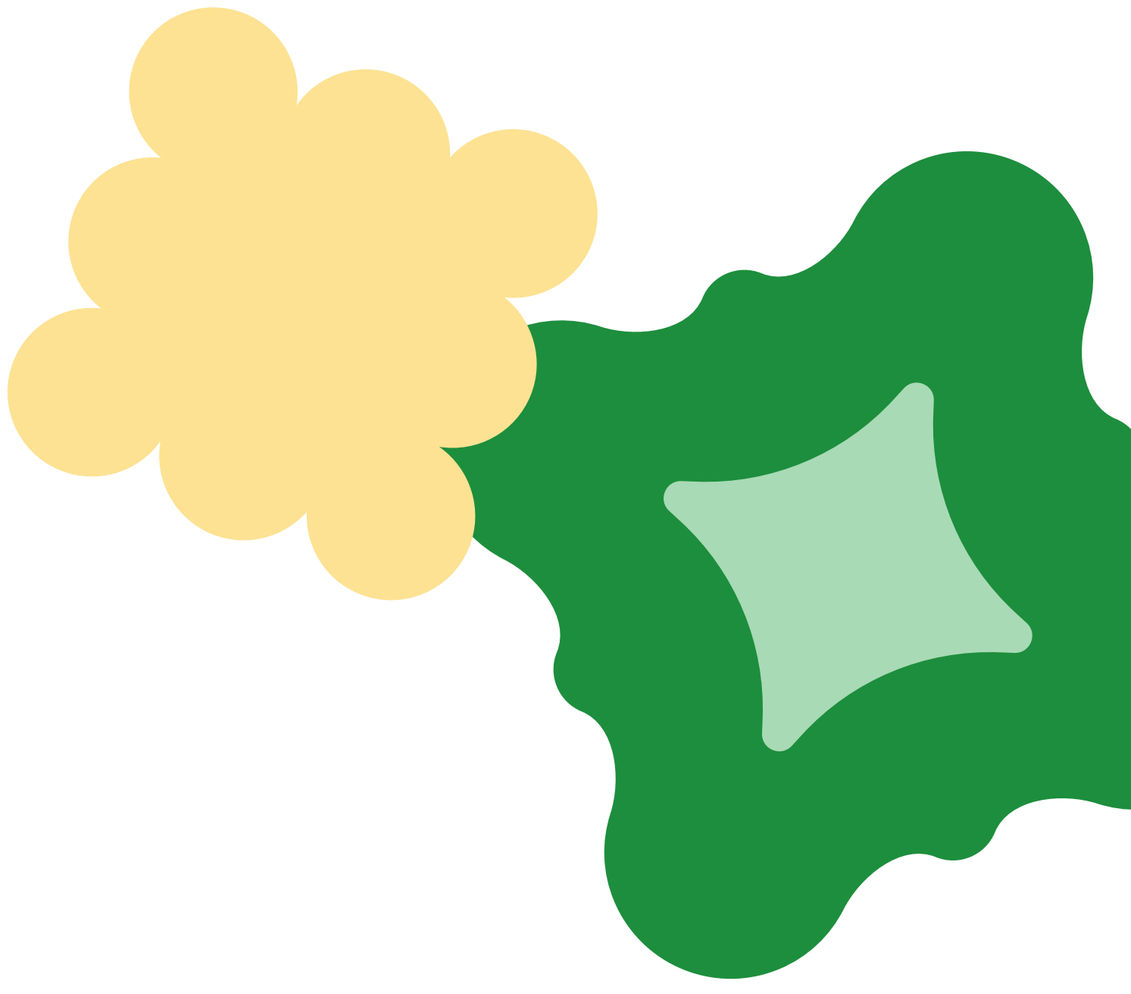
Reference: ADK Docs - Loop Agents

4. Choosing the right workflow type

Type	Pattern	Use When
Sequential	A ⟶ B ⟶ C	Fixed order, each step needs previous output
Parallel	A + B + C	Independent tasks, want speed
Loop	(A ⟶ B) × N	Iterative refinement, quality loops

Quick decision guide:

- Do steps depend on each other? ⟶ **Sequential**
- Can steps run independently? ⟶ **Parallel**
- Need to repeat until good enough? ⟶ **Loop**



Hands-on example

Example 1

Sequential pipeline

A simple 3-step request processing pipeline.

Python

```
"""
Sequential Pipeline Example
Each step depends on the previous step's output.
"""

from google.adk.agents import LlmAgent, SequentialAgent

# Step 1
intake = LlmAgent(
    model='gemini-2.5-flash',
    name='intake',
    output_key='request',
    instruction='Extract and summarize the user request'
)

# Step 2
analyzer = LlmAgent(
    model='gemini-2.5-flash',
    name='analyzer',
    output_key='analysis',
    instruction='Analyze this request: {request}'
)

# Step 3
responder = LlmAgent(
    model='gemini-2.5-flash',
    name='responder',
    instruction='Respond to the user based on: {analysis}'
)

root_agent = SequentialAgent(
    name='request_pipeline',
    sub_agents=[intake, analyzer, responder]
)
```

Example 2

Parallel research

Gather information from two sources at once.

Python

```
"""
Parallel Research Example
Two researchers run simultaneously.
"""

from google.adk.agents import LlmAgent, ParallelAgent, SequentialAgent

# Research source 1
researcher_benefits = LlmAgent(
    model='gemini-2.5-flash',
    name='benefits_researcher',
    output_key='benefits',
    instruction='Research the benefits of renewable energy'
)

# Research source 2
researcher_challenges = LlmAgent(
    model='gemini-2.5-flash',
    name='challenges_researcher',
    output_key='challenges',
    instruction='Research the challenges of renewable energy'
)

# Parallel research
parallel_research = ParallelAgent(
    name='research',
    sub_agents=[researcher_benefits, researcher_challenges]
)

# Summarize (after parallel completes)
summarizer = LlmAgent(
    model='gemini-2.5-flash',
    name='summarizer',
    instruction='Summarize: Benefits: {benefits}, Challenges: {challenges}'
)

# Full pipeline: parallel research → summarize
root_agent = SequentialAgent(
    name='research_pipeline',
    sub_agents=[parallel_research, summarizer]
)
```

Test it: `adk web` → "Tell me about renewable energy"

Example 3

Refinement loop

Iterate to improve quality.

Python

```
"""
Refinement Loop Example
Check quality and improve, up to 3 times.
"""

from google.adk.agents import LlmAgent, LoopAgent, SequentialAgent

# Initial draft
drafter = LlmAgent(
    model='gemini-2.5-flash',
    name='drafter',
    output_key='draft',
    instruction='Write a short paragraph about the given topic'
)

# Quality check
checker = LlmAgent(
    model='gemini-2.5-flash',
    name='checker',
    output_key='feedback',
    instruction='Rate this draft 1-10 and give feedback: {draft}'
)

# Improve
improver = LlmAgent(
    model='gemini-2.5-flash',
    name='improver',
    output_key='draft',  # Updates the draft
    instruction='Improve the draft based on feedback: {feedback}'
)

# Refinement loop
refinement = LoopAgent(
    name='refinement',
    sub_agents=[checker, improver],
    max_iterations=2  # Check and improve twice
)

# Full pipeline: draft → refine
root_agent = SequentialAgent(
    name='writing_pipeline',
    sub_agents=[drafter, refinement]
)
```

Test it: `adk web` → “Write about artificial intelligence”

Key takeaways

Workflow agents give you precise control over how your agents execute. Unlike LLM-driven delegation where the model decides routing, workflow agents follow fixed, predictable patterns. This makes them ideal for structured processes where you need guaranteed execution order.

What workflow agents provide:

- Deterministic, predictable execution patterns
- No LLM decision-making about execution order
- Three types for different needs

The three workflow agents:

- **SequentialAgent** - Run in order (A → B → C)
- **ParallelAgent** - Run together (A + B + C)
- **LoopAgent** - Repeat (A → B, repeat)

Data passing between agents:

- Use `output_key` to save output to state
- Use `{key}` in instructions to read from state

When to use each:

- Sequential: Steps depend on each other
- Parallel: Steps are independent (faster)
- Loop: Need to iterate/refine